

MODELISER-RESOUDRE

Intelligence artificielle



Initiation au MACHINE LEARNING et algorithmes associés

C.P.G.E.
du lycée
ROLAND GARROS

Cours proposé par
D. GONCALVES

SOMMAIRE :

INITIATION AU MACHINE LEARNING ET ALGORITHMES ASSOCIES2

1. Histoire de l'IA	2
2. Type de problématiques et solutions associées du Machine Learning	2
3. Méthodologie du machine Learning	3
4. La bibliothèque Sklearn (Scikit-learn)	5
5. Algorithmes fondamentaux utilisés en Machine Learning	5

Initiation au machine learning et algorithmes associés

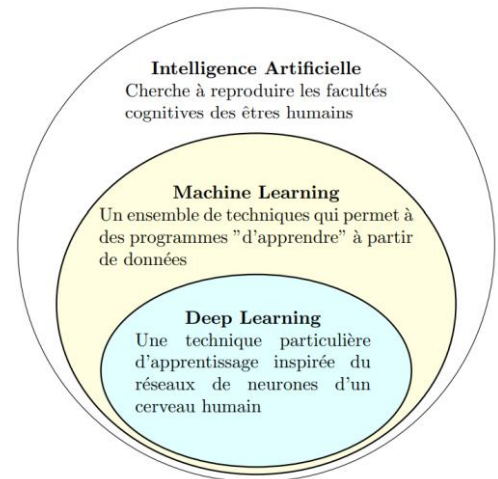
INTRODUCTION :

Bien que ce cours se nomme **intelligence artificielle (IA)**, nous nous limiterons ici à présenter les méthodes dites du **Machine Learning** et plus particulièrement celles liées à l'**apprentissage supervisé**. On s'intéressera notamment au **Deep Learning** très utilisée aujourd'hui.

Ces méthodes concernent pratiquement toutes les applications opérationnelles de l'IA actuelles : assistance humaine (Alexa, Google Home, Chat JPT, etc...), jeux (Deep Blue, AlphaGo, etc...), moteurs de recherche (Google, Bing ou autre), reconnaissance faciale, etc...

Le concept général consiste à fournir à une machine un grand nombre d'exemples est de lui demander de prédire sur cette base, la réponse la plus probable à une question posée. Ce dernier point constitue la pierre angulaire du Machine Learning que l'on pourrait définir comme suite :

Le Machine Learning consiste à donner à une machine la capacité d'apprendre au travers d'exemples, sans la programmer de façon explicite...

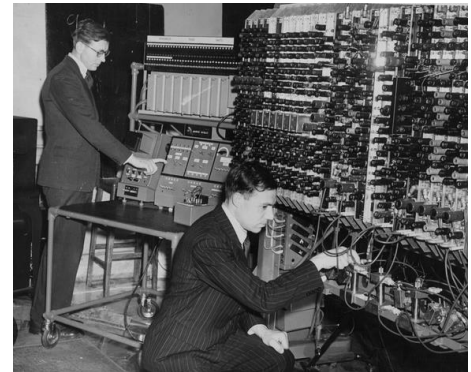


1. HISTOIRE DE L'IA :

Avant d'entrer dans le vif du sujet, nous allons découvrir que ce concept est loin d'être nouveau...

Voyons cela au travers d'une petite vidéo proposée par un **Datascientist** Guillaume SAINT CIRGUE que vous pouvez retrouver en tapant le lien :

https://www.youtube.com/watch?v=XUFLq6dKQok&list=PLO_fdPEVlfKoanjvTJblbd9V5d9Pzp8Rw



Machine de Turing

2. TYPE DE PROBLEMATIQUES ET SOLUTIONS ASSOCIEES DU MACHINE LEARNING :

2.1 Apprentissages supervisé et non supervisé :

On distingue dans les algorithmes de l'IA deux catégories : l'apprentissage supervisé et l'apprentissage non supervisé. La différence entre ces deux types est fondamentale...

- **L'apprentissage supervisé** : A partir d'un ensemble de données, appelé **Dataset** contenant des couples entrées-sortie, on précise à la machine les paramètres d'entrée (**Features**) à considérer afin qu'elle puisse prédire la sortie (**Target ou labels**) probable.

Exemple : prix d'un appartement à Paris :

Sortie	Entrées			
Prix (euros)	Surface (m ²)	Nombre de chambres	Arrondissement	Qualité (1 à 10)
364000	35	2	12	8
560000	57	2	14	5
1850000	112	3	7	7
282000	22	1	3	6
526000	32	2	5	7
...	...	...	...	...

On comprend bien entendu que plus le nombre d'exemples sera important et plus la prédiction sera bonne !

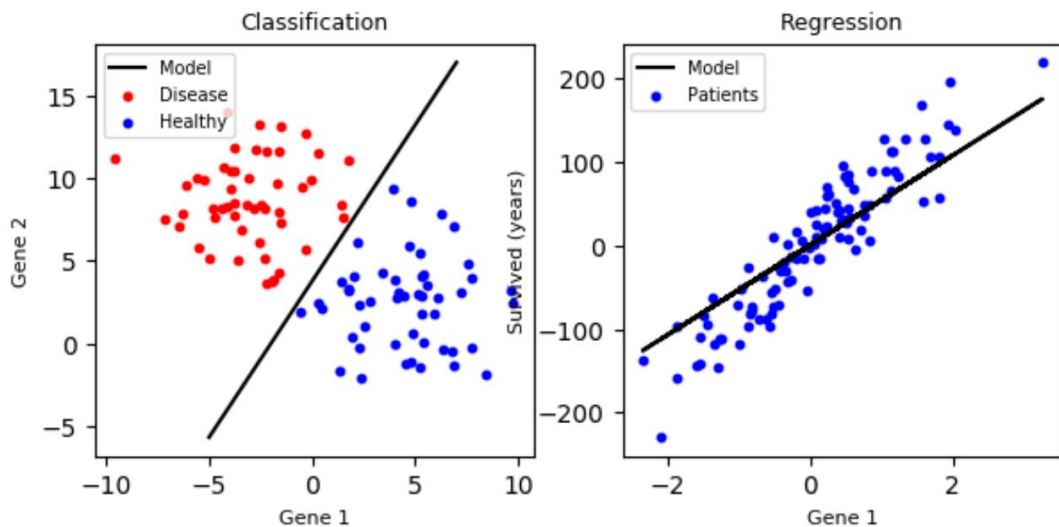
- **L'apprentissage non supervisé** : A contrario, les algorithmes non supervisés n'intègrent pas la notion d'entrée-sortie. Dans ce cas, les algorithmes cherchent à organiser les données en groupes observables.

Ce type d'apprentissage n'est pas au programme !

2.2 Problématique de régression ou de classification (apprentissage supervisé) :

- **Les problématiques de régression** : La sortie peut prendre une infinité de valeurs dans un ensemble continu de réels. L'exemple ci-dessus est typiquement un exemple de problématique de régression ; le prix est bien un réel dans un ensemble continu de valeurs.

• **Les problématiques de classification** : On cherchera à déterminer une prévision d'appartenance de la sortie à une catégorie (*class labels*). Un cas particulier est celui de la classification binaire ; exemple : on cherche à savoir si une cellule est cancéreuse ou non, à partir de certains paramètres (taille de la cellule, du noyau, etc...).



3. METHODOLOGIE DU MACHINE LEARNING :

3.1 Le prétraitement du Dataset :

La première étape consiste à représenter mathématiquement les données d'entrée et de sortie. On distinguera alors deux types de données :

• **Les données quantitatives** : Elles ont une représentation mathématique directe. Elles peuvent être continues (mesure d'une grandeur) ou discrètes (âge, nombre d'enfants, facteurs RGB d'une couleur, etc...).

• **Les données qualitatives** : A contrario, elles n'ont pas de représentation mathématique directe. Elles apportent une information sur une caractéristique (sexe, race d'un chien, vrai/faux, etc...). On pourra leur attribuer des valeurs numériques pour faciliter le traitement, mais pour autant, et contrairement aux données quantitatives, on ne pourra pas définir de relation d'ordre entre ces valeurs.

A partir du moment où on a donné une représentation numérique à notre *dataset*, on va pouvoir représenter ce dernier sous forme matricielle. On aura ainsi deux matrices : une matrice colonne **Y** (ou vecteur) contenant les *m valeurs cibles* et une matrice **X** de dimensions $m \times n$ contenant les *n paramètres des m exemples*.

Dataset (x, y)

vecteur target $y \in \mathbb{R}^{m \times 1}$

$$y = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \dots \\ y^{(m)} \end{pmatrix}$$

y	x_1	x_2	x_3	...	x_n
$y^{(1)}$	$x_1^{(1)}$	$x_2^{(1)}$	$x_3^{(1)}$	...	$x_n^{(1)}$
$y^{(2)}$	$x_1^{(2)}$	$x_2^{(2)}$	$x_3^{(2)}$	...	$x_n^{(2)}$
$y^{(3)}$	$x_1^{(3)}$	$x_2^{(3)}$	$x_3^{(3)}$	...	$x_n^{(3)}$
...	...	...	...	...	...
$y^{(m)}$	$x_1^{(m)}$	$x_2^{(m)}$	$x_3^{(m)}$	...	$x_n^{(m)}$

matrice features $X \in \mathbb{R}^{m \times n}$

$$X = \begin{pmatrix} x_1^{(1)} & \dots & x_n^{(1)} \\ \vdots & \ddots & \vdots \\ x_1^{(m)} & \dots & x_n^{(m)} \end{pmatrix}$$

Si on reprend nos appartements à Paris, en se limitant aux cinq exemples donnés, on aura alors :

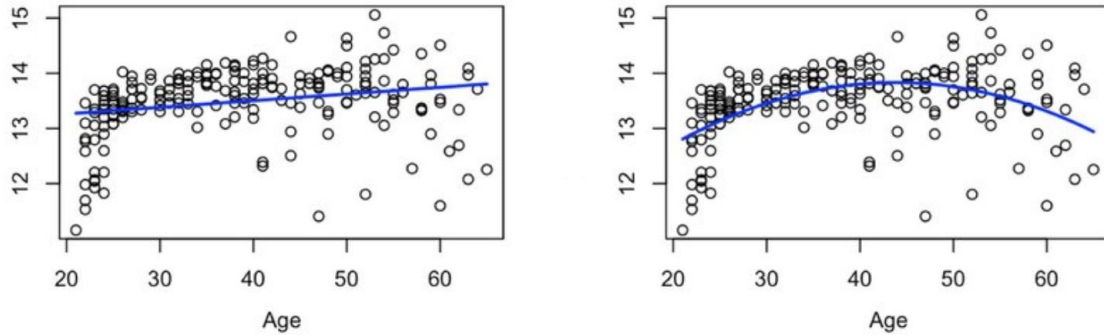
$$Y = \begin{bmatrix} 364000 \\ 560000 \\ 1850000 \\ 282000 \\ 526000 \end{bmatrix}, \quad X = \begin{bmatrix} 35 & 2 & 12 & 8 \\ 57 & 2 & 14 & 5 \\ 112 & 3 & 7 & 7 \\ 22 & 1 & 3 & 6 \\ 32 & 2 & 5 & 7 \end{bmatrix} \quad \text{et} \quad X_{norm} = \begin{bmatrix} 0,14 & 0,5 & 0,82 & 1 \\ 0,39 & 0,5 & 1 & 0 \\ 1 & 1 & 0,36 & 0,67 \\ 0 & 0 & 0 & 0,33 \\ 0,11 & 0,5 & 0,18 & 0,67 \end{bmatrix} \quad \text{avec} \quad x_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

En général, les données d'entrée sont normalisées, c'est-à-dire mises à la même échelle, le plus souvent entre 0 et +1. En effet, si les différences entre les valeurs sont trop importantes, les algorithmes peuvent peiner à converger ou privilégier une colonne par rapport à une autre.

Une fois le Dataset représenté sous forme d'une matrice normalisée, on va le découper au hasard en deux parties : *un jeu dit d'entraînement* (environ 80% des données) et *un jeu de test* (le reste).

3.2 Le choix du modèle :

Que ce soit pour un problème de régression ou de classification, il s'agit maintenant de définir un modèle.



Pour l'exemple ci-dessus, la répartition est-elle plutôt décrite par une droite ou une courbe ?

Ce choix sera fondamental dans la performance de notre algorithme ; nous reviendrons sur cette notion plus loin...

3.3 Evaluer la qualité du modèle :

- Dans les problématiques de régression : On va comparer les prédictions effectuées que l'on notera $\hat{y}_i$

aux vraies cibles, notées y_i , modélisée par la **distance euclidienne** :

$$d_i = \sqrt{(\hat{y}_i - y_i)^2}$$

Ceci permettra de définir la **fonction coût** ou **erreur quadratique moyenne** :

$$J = \frac{1}{m} \sum_{i=1}^m d_i^2$$

Le coefficient de Pearson, appelé aussi coefficient de détermination est un classique dans le calcul statistique...

Ou encore le **coefficient de Pearson** :

$$R^2 = 1 - \frac{\sum_{i=1}^m (\hat{y}_i - y_i)^2}{\sum_{i=1}^m (\hat{y}_i - \bar{y})^2}$$

- Dans les problématiques de classification : Elle sera définie par la **justesse (accuracy)**, notée $A = \frac{V}{N}$,

avec V le nombre de prédictions correctes et N le nombre de prédictions.

3.4 Phase d'apprentissage :

Le concept de base de l'**apprentissage** en IA consiste à **chercher à minimiser l'erreur commise par le modèle**.

Prenons un exemple simple d'un modèle linéaire du type : $\hat{y}_i = a.x_i + b$ (illustré plus loin...)

La fonction coût s'écrit alors : $J(a,b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 = \frac{1}{2m} \sum_{i=1}^m (a.x_i + b - y_i)^2$

Le $\frac{1}{2}$ est un artifice calculatoire...

On peut remarquer que **la fonction coût est quadratique**, donc qu'elle **n'admet qu'un minimum**. On comprend alors que le trouver, revient à minimiser l'erreur commise, donc à donner la meilleure réponse possible !

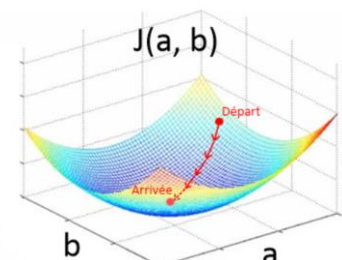
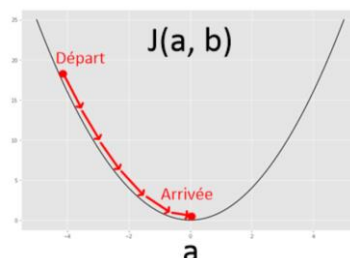
Deux méthodes s'offrent à nous :

- La **méthode des moindres carrés** (ou **équations normales**) : cette méthode matricielle n'est pas toujours applicable ce qui limite son utilisation. Nous reviendrons sur ce point plus loin...

- La **descente de gradient** : Elle consiste à chercher par itération à minimiser les gradients de a et de b .

Cette méthode n'est pas au programme !

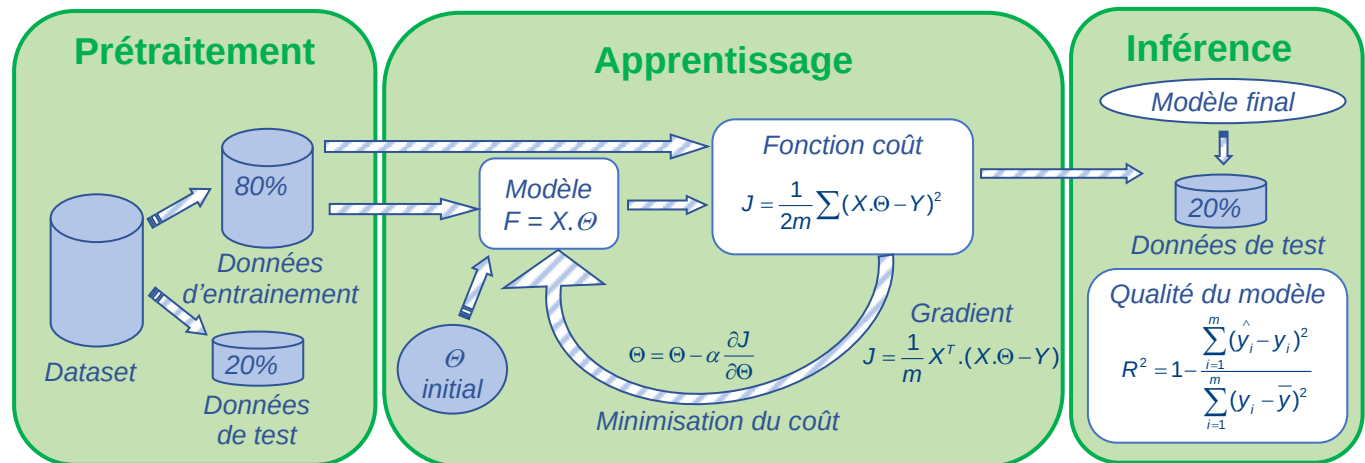
Illustration de la descente de gradient ci-contre :



3.6 Phase d'inférence :

La dernière étape consiste maintenant à tester notre modèle sur les 20% des données qui n'ont pas été utilisées pour l'apprentissage et de tester par exemple le coefficient de Pearson sur ces données...

L'ensemble de la méthode peut être résumée par le schéma suivant :



4. LA BIBLIOTHÈQUE SCIKIT-LEARN (SKLEARN) :

La bibliothèque **Scikit-learn** propose en Python sous le nom : **sklearn**, à peu près tout ce qu'il est possible de faire en **Machine Learning** aujourd'hui. Nous verrons plus loin au-travers d'exemples, quelques applications classiques...

5. ALGORITHMES FONDAMENTAUX UTILISÉS EN MACHINE LEARNING :

5.1 Algorithmes de régression :

• **Régression linéaire univariée** : La sortie y ne dépend que d'un seul paramètre x ; $y_i = f(x_i)$. Si ce type de problématique est intéressante pour comprendre les mécanismes d'apprentissage car il permet une représentation simple dans le plan, elle sera en pratique très rare ; un **Label** dépendant rarement d'une seule **Feature**.

Illustrons malgré tout cette méthode...

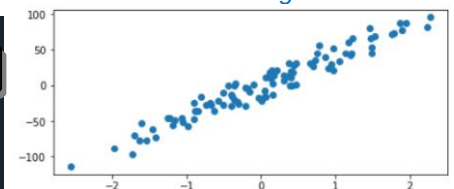
On génère tout d'abord des données linéaires bruitées...

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression

# génération des données (1 paramètre par sortie)
x, y = make_regression(n_samples=100, n_features=1, noise=10)
y = y.reshape(y.shape[0], 1) # reformatage de y
plt.scatter(x, y) # tracé des données générées
```

Univariée !

Tracé des données générées...



make_regression renvoie une dimension incomplète. On reformate y...

On souhaite un modèle linéaire du type : $\hat{y}_i = a \cdot x_i + b$

On vectorise le calcul matriciel : $Y = X \cdot \Theta$ où Θ est le vecteur $\Theta = \begin{bmatrix} a \\ b \end{bmatrix}$ et X la matrice $X = \begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix}$

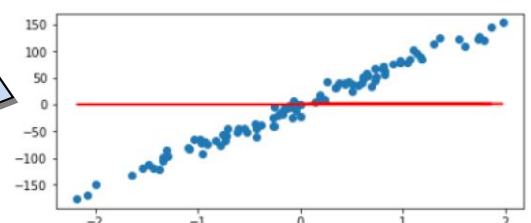
```
#vectorisation du modèle ax+b avec numpy
X = np.hstack((x, np.ones(x.shape)))
theta = np.random.randn(2, 1) # les paramètres a et b sont aléatoires
```

On définit ensuite le modèle et on le trace...

```
#modèle
def model(X, theta):
    return X.dot(theta)

plt.plot(x, model(X, theta), c='r')
```

Notre modèle n'est pas très performant ! Normal, il est aléatoirement...



On écrit notre fonction coût à partir de nos matrices X , Y et Θ ...

$$J = \frac{1}{2m} \sum_{i=1}^m (\hat{y} - y_i)^2 = \frac{1}{2m} \sum_{i=1}^m \underbrace{(a.x_i + b)}_{X.\Theta} - y_i)^2 \quad \text{qui s'implémente comme ci-dessous :}$$

```
#fonction cout
def cost_function(X,y,theta):
    m = len(y)
    return 1/(2*m)*np.sum((model(X,theta)-y)**2)
```

Rappel : le $\frac{1}{2}$ n'est qu'un artifice calculatoire pour simplifier l'écriture de la dérivée...

On cherche le minimum de la fonction coût en calculant sa dérivée en fonction des paramètres a et b .

$$\frac{\partial J(a,b)}{\partial a} = \frac{1}{m} \sum_{i=1}^m x_i (a.x_i + b - y_i) \quad \text{et} \quad \frac{\partial J(a,b)}{\partial b} = \frac{1}{m} \sum_{i=1}^m (a.x_i + b - y_i)$$

Comme auparavant, nous pouvons vectoriser ces dérivées :

Soit le vecteur : $\frac{\partial J(\Theta)}{\partial \Theta} = \begin{bmatrix} \frac{\partial J(a,b)}{\partial a} \\ \frac{\partial J(a,b)}{\partial b} \end{bmatrix}$, alors : $\frac{\partial J(\Theta)}{\partial \Theta} = \frac{1}{m} X^T (X.\Theta - Y)$, avec : $X^T = \begin{bmatrix} x_0 & x_1 & \dots & x_n \\ 1 & 1 & \dots & 1 \end{bmatrix}$

On trouve les minimums de $J(\Theta)$ en cherchant les zéros de $\frac{\partial J(\Theta)}{\partial \Theta}$ par **la méthode des moindres carrés**.

Ceci peut s'écrire :

$$\frac{\partial J(\Theta)}{\partial \Theta} = \frac{1}{m} X^T (X.\Theta - Y) = 0 \Leftrightarrow X^T.X.\Theta - X^T.Y = 0$$

Cela donne :

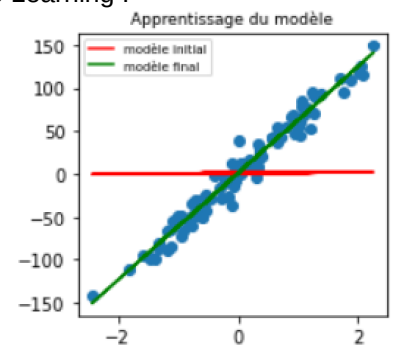
$$\Theta = (X^T.X)^{-1}.X^T.Y$$

Ce calcul a priori simple nécessite cependant d'inverser la matrice $X^T.X$, ce qui peut poser un problème. En effet, d'une part, ce ne sera pas toujours possible et d'autre part, cela deviendra très vite compliqué (même pour une machine) pour de grandes dimensions de X , ce qui est toujours le cas en Machine Learning !

En pratique, on aura tendance à utiliser cette méthode en début de projet sur un Dataset réduit, puis de privilégier la descente de gradient (hors programme).

Le résultat de l'apprentissage dans notre cas est traduit sur le tracé ci-contre et l'implémentation donnée ci-dessous :

```
def moindre_carre(X,y):
    X_T=np.transpose(X)
    D=np.linalg.inv(np.dot(X_T,X))
    theta=np.dot(np.dot(D,X_T),y)
    return theta
```

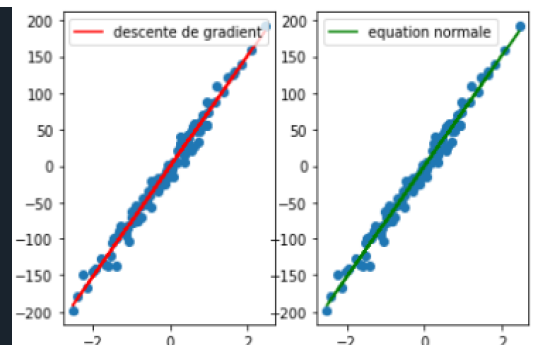


En utilisant la bibliothèque *ScikitLearn* avec le même jeu de données on a le code ci-dessous...

```
# #modèle sklearn descente de gradient
model_2 = SGDRegressor(max_iter=1000, eta0=0.01)
model_2.fit(x,y)
print('Coeff de Pearson modele2=', model_2.score(x,y))

# #modèle sklearn équation normale
model_3 = LinearRegression()
model_3.fit(x,y)
print('Coeff de Pearson modele3=', model_3.score(x,y))

plt.subplot(1,2,1)
plt.scatter(x,y) #tracé des données générées
#tracé du modèle généré par la descente de gradient
plt.plot(x,model_2.predict(x), c='r', label='descente de gradient')
plt.legend()
plt.subplot(1,2,2)
plt.scatter(x,y) #tracé des données générées
#tracé du modèle généré par la méthode des moindres carrés
plt.plot(x,model_3.predict(x), c='g', label='equation normale')
plt.legend()
```



Coeff de Pearson modele2= 0.9761818933325939
Coeff de Pearson modele3= 0.9761823400066219

Erreur de moins de 3%...

• **Régression linéaire multivariée** : La régression multivariée n'est en fait qu'une généralisation de la méthode précédente. Dans *Scikit-learn*, l'implémentation sera strictement la même.

• **Régression polynomiale** : L'idée est la même, sauf qu'au lieu de définir une droite pour modèle, on prendra un polynôme de degré d . On pourrait d'ailleurs implémenter la régression polynomiale en reprenant le code vu précédemment et en donnant à la matrice X la dimension $d + 1$ (ne pas oublier la colonne de 1) et au vecteur Θ $d + 1$ termes. Cela donnerait pour un degré 2 :

$$X = \begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ \vdots & \vdots & \vdots \\ x_n^2 & x_n & 1 \end{bmatrix} \text{ et } \Theta = \begin{bmatrix} a \\ b \\ c \end{bmatrix} \text{ qui s'implémentera simplement :}$$

Tout le reste du code est identique...

```
X=np.hstack((x,np.ones(x.shape)))
X=np.hstack((x**2,x))
theta = np.random.randn(3,1)
```

Là encore, *Scikit-learn* propose une méthode pré-implémentée que nous allons privilégier ici...

On donne ci-dessous un exemple pour lequel on a créé artificiellement un polynôme bruité d'ordre 3. On laisse ensuite la machine apprendre sur la base de polynômes de différents degrés (tracés en face du code)...

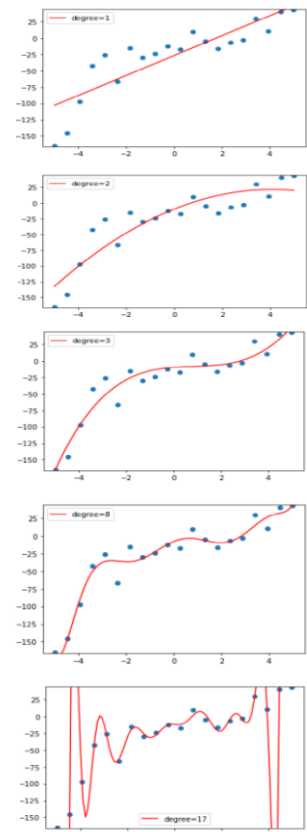
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.datasets import make_regression
from sklearn.preprocessing import PolynomialFeatures

#génération d'une courbe d'ordre 3 bruitée
np.random.seed(0)
x = np.linspace(-5, 5, 20)
noise = np.random.normal(-20, 20, 20)
y=x**3-2*x**2+x+noise
plt.scatter(x,y)#tracé des points

x = x.reshape(-1,1)#transforme x en matrice colonne
x_grid = np.linspace(-5,5,100)
x_grid = x_grid.reshape(-1,1)

d=3
poly=PolynomialFeatures(degree=d)#définition du modèle
x_poly = poly.fit_transform(x)#apprentissage
linReg = LinearRegression()
linReg.fit(x_poly,y)#création de la courbe guide
#application du modèle et de la courbe guide à x_grid
x_grid_poly = poly.fit_transform(x_grid)
pred_grid = linReg.predict(x_grid_poly)

plt.plot(x_grid, pred_grid, color = 'r', label = 'degree='+str(d))
plt.ylim(min(y)-1, max(y)+1)
plt.legend()
```

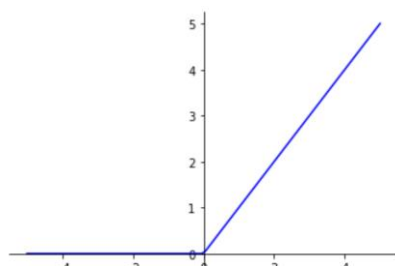


Sans trop de surprise, le modèle d'ordre 3 semble le meilleur ! Par contre, on peut remarquer le risque que fait prendre un modèle d'ordre trop élevé avec l'apparition de fortes oscillations. On parle alors de **surapprentissage**.

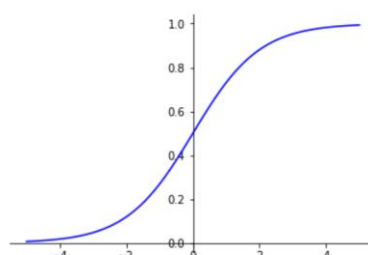
En pratique, c'est la connaissance du phénomène source des données qui permettra le choix d'un bon modèle !

5.2 Algorithmes de classification :

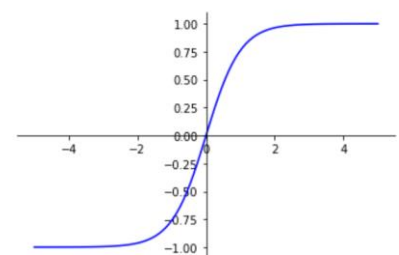
• **Méthode d'apprentissage** : A l'inverse de la régression où le lien entre les prédictions et le modèle est continu, dans un problème de classification, la question sera de type binaire donc discontinu : l'échantillon appartient-il à cette catégorie ou non ? A l'instar de la **fonction Heaviside** (que nous n'utiliserons pas en IA), plusieurs fonctions permettent d'évaluer de façon continue un événement discontinu. Les trois principales sont :



Fonction ReLU – Rectified Linear Unit



Fonction Sigmoide



Fonction tanh

Nous nous limiterons à présenter la **fonction sigmoïde** : $\sigma(x) = \frac{1}{1+e^{-x}}$ nommée aussi **fonction Logistique**.

Cette fonction va ainsi nous renvoyer la probabilité qu'une donnée appartienne ou non à telle catégorie !

Il est alors possible de chercher à minimiser l'erreur commise par le modèle. Pour cela, on peut notamment utiliser la fonction coût appelée **Log Loss** et définie par :

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \ln(a_i) + (1 - y_i) \cdot \ln(1 - a_i)$$

Nous ne détaillerons pas cette méthode ici !

Là encore la bibliothèque *Scikit-learn* propose une méthode appelée **Régression Logistique** permettant d'entraîner un modèle sur la base de la fonction sigmoïde. Son implémentation est la suivante :

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression() #définition du modèle
model.fit(X_train, y_train) #apprentissage du modèle
y_pred = model.predict(X_test) #prediction
```

Ici, on imagine qu'on a un jeu d'entraînement ($X_{\text{train}}, y_{\text{train}}$) et qu'on cherche à prédire la sortie y_{pred} d'un jeu de test de X_{test} ...

• **Qualité du modèle** : Comme définit plus haut, dans un problème de classification, on utilise en général la **justesse (Accuracy)** : proportion de prédictions correctes sur le nombre de prédictions.

Scikit-learn permet cette évaluation en tapant :

```
from sklearn.metrics import accuracy_score

accuracy_score(y_true, y_pred)
```

y_{true} sont les vraies sorties du jeu de test, X_{test} et y_{pred} ses prédictions...

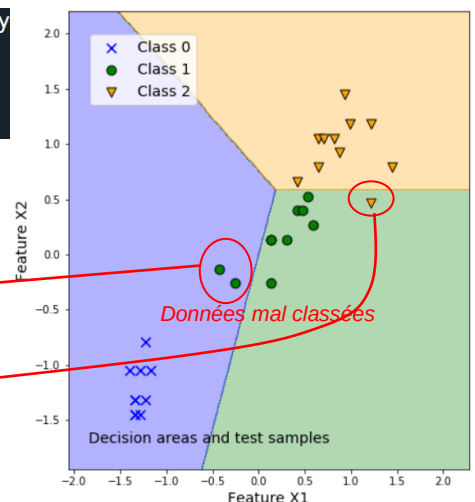
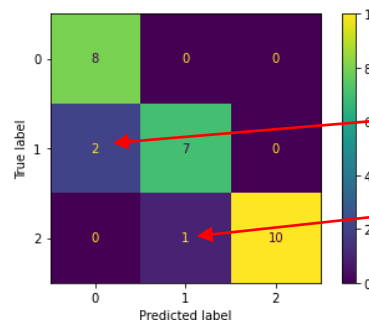
Il est également possible de visualiser à l'aide *Scikit-learn*, la **matrice de confusion** qui représentera la différence entre les prédictions et les vraies catégories (ou **true label**). On illustre ci-dessous cette matrice avec deux jeux contenant chacun 28 données regroupées en trois catégories (ou **Class** 0, 1, 2 établies sur deux paramètres (ou **Features**) X_1 et X_2 . Le jeu y_{true} correspond au jeu de test et le jeu y_{pred} est issu d'un algorithme de classification. On retrouve la justesse en faisant le rapport de la somme des termes de la diagonale sur le nombre total de prédictions. **Plus cette matrice est diagonale et plus le modèle est bon !**

On obtient cette matrice sous Scikit learn en tapant :

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot
```

Dans cet exemple, trois données sont mal classées : deux de la Class 1, classées dans la Class 0 et une de la Class 2, classée dans la Class 1. La justesse vaut donc ici : 25/28



5.3 Algorithmes des k plus proches voisins :

L'algorithme **k-NN (k-nearest neighbors)** est une méthode d'apprentissage supervisée aussi bien utilisée dans les problèmes de régression que de classification. Son fonctionnement peut être assimilé à l'analogie suivante : « dis-moi qui sont tes voisins et je te dirai qui tu es ».

Cet algorithme ayant déjà fait l'objet d'un développement en cours d'informatique, nous nous contenterons ici de rappeler quelques principes ainsi que son implémentation dans *Scikit-learn*.

• **Dans un problème de régression** : c'est la moyenne (ou la médiane) des variables y_i des k plus proches voisins qui servira pour la prédiction ;

- **Dans un problème de classification** : on utilisera la valeur du groupe majoritaire des **k-NN**.

Avec *Scikit-learn*, on peut ainsi obtenir une prévision basée sur les **k-NN** en régression ou classification en tapant :

```
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor

model = KNeighborsClassifier() #ou KNeighborsRegressor
model.fit(X,y) #apprentissage du modèle KNN

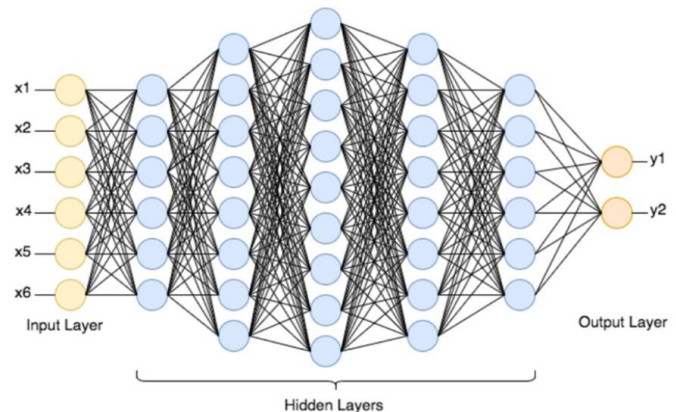
y_pred = model.predict(y_test) #prediction sur le jeu de test
```

5.4 Perceptron et réseaux de neurones :

• **Concept général** : basé sur l'observation des neurones naturels, **les réseaux de neurones artificiels** sont constitués de multiples neurones appelés **Perceptron**. Ces derniers sont organisés en couches plus ou moins nombreuses (ou profondes) en fonction de la complexité de la modélisation recherchée.



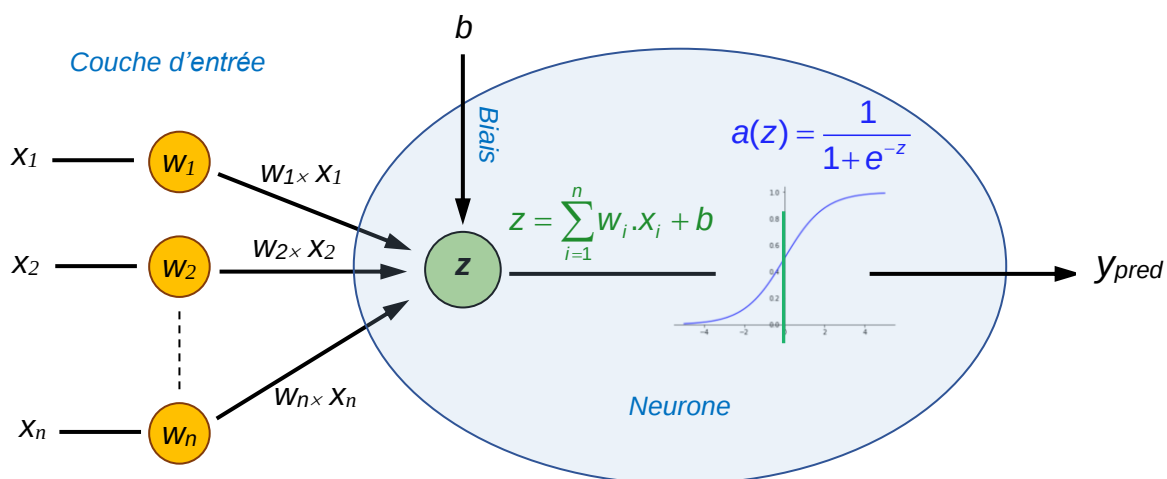
Réseau de neurones naturels



Réseau de neurones artificiels

Dans chaque réseau de neurones, on distingue la couche d'entrée (**input layer**) qui prend en argument les paramètres d'entrée à considérer, la couche de sortie (**output layer**) qui produit les sorties attendues, et différentes couches cachées. Lorsqu'un réseau de neurones possède plusieurs couches de neurones cachées, on parle alors d'**apprentissage profond (Deep learning)**.

- **Le perceptron** : c'est l'unité élémentaire d'un réseau de neurones...

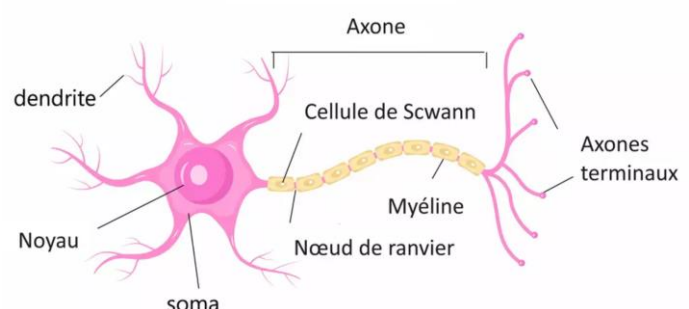


Souvent comparé aux neurones naturels (illustré ci-contre), son fonctionnement, purement mathématique, est très différent...

La fonction **z** fait ainsi intervenir des **poids** w_i (positifs ou négatifs) affectés à chaque grandeur d'entrée x_i et un **biais**, noté **b**.

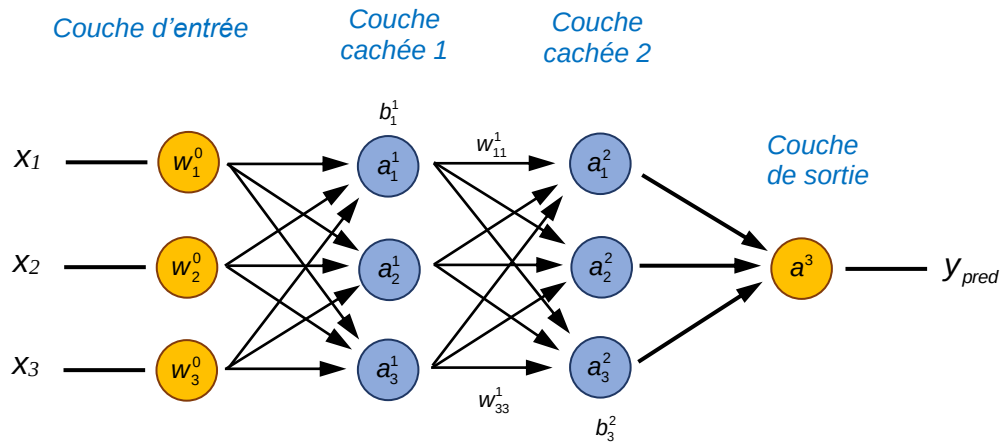
Le mécanisme de transmission est ainsi le résultat d'une fonction mathématiques linéaire **z** suivie d'une fonction d'activation non-linéaire **a(z)** (la fonction *Sigmoid* par exemple ici).

Neurone naturel



L'apprentissage d'un perceptron reprendra les mêmes principes vus précédemment. En cherchant à minimiser la fonction coût, on définira les différents poids et biais du neurone.

- **Réseaux de neurones** : dans un réseau de neurones, les sorties d'une couche vont alimenter les entrées de la couche suivante. On illustre ci-dessous un réseau à deux couches cachées de trois neurones chacune, trois paramètres d'entrée et une sortie.



Là encore les concepts d'apprentissage resteront identiques. On définira les différents poids et biais en cherchant à minimiser la fonction coût. Pour faire cela, la méthode particulièrement adaptée aux réseaux est la **back-propagation**. Sans rentrer dans le détail, il s'agit d'une méthode itérative qui consiste à évaluer l'erreur de la sortie et de propager vers l'arrière la minimisation de la fonction coût en mettant à jour les différents poids et biais.

- **Utilisation sous Python** : il existe plusieurs bibliothèques permettant de mettre en œuvre les réseaux de neurones, ainsi que les autres méthodes de l'IA d'ailleurs sous Python. Les plus connues actuellement sont : **Tensorflow**, **Pytorch**, **Keras** et bien entendu **Scikit-learn**. Afin de ne pas surcharger ce cours, et comme pour les autres méthodes vues précédemment, nous nous limiterons ici à présenter la bibliothèque **Scikit-learn** ; les autres pourront être abordées en TP.

```
from sklearn.neural_network import MLPClassifier, MLPRegressor

# Création du modèle
model = MLPClassifier(hidden_layer_sizes=(3, 3), activation='logistic', max_iter=1000, )

# Alternative pour un problème de régression
model = MLPClassifier(hidden_layer_sizes=(3, 3), activation='relu', max_iter=1000)

model.fit(X_train, y_train) #apprentissage du modèle
model.score(X_train, y) #score du modèle
```

Depuis un jeu d'entraînement (X_{train}, y_{train})...

Ici on limite le nombre d'itérations et on impose la fonction d'activation...